



Faculty of Engineering and Technology — Computers & Intelligent Systems Engineering



Together

An AI-Based Sign-Language Translator (ASL & ArSL)

Graduation Project Defense

Abdelfattah Moustafa

20225889

Michael Abdallah

20213601

Ahmed Mohamed Nagib

20170423

Ahmed Ashraf Shawareb

20180097

Supervisor: Prof. Medhat Awadallah

July 2026

Four speakers, one system

Speaker 1 — The problem & the field

Why sign-language translation matters, what exists today, and the gap Together fills.

Speaker 2 — Architecture & the ASL model

System design, the browser-to-server pipeline, and the 250-class Squeezeformer-style recognizer.

Speaker 3 — ArSL, language & the platform

The Egyptian ArSL recognizer, the LLM translation layer, sign synthesis, and the eight app modules.

Speaker 4 — Evaluation & live demo

Recognition and translation results, latency, limitations — then Together running live.

Hearing loss at global scale

1.5 billion+

people live with some degree of hearing loss — and for millions of them, a signed language is their first and most natural language.

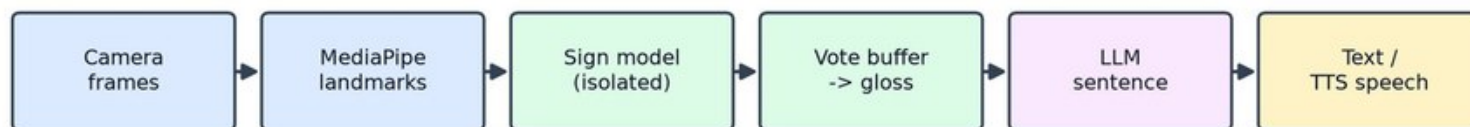
World Health Organization, Deafness and Hearing Loss fact sheet, 2024

A translation gap, not a linguistic deficit

- **Full languages.** ASL and ArSL have their own grammar, syntax and space — not signed English or Arabic.
- **Two directions needed.** A real conversation requires sign → speech and speech → sign at the same time — interpreters cannot scale to that.

Figure 1.2 — The bidirectional translation pipelines

Sign -> Text / Speech



Text / Speech -> Sign



Why current systems fall short

1 One-way streets

Systems either read sign or produce sign
— almost never both in one platform.

2 High-resource bias

Research concentrates on ASL; Arabic
Sign Language is critically under-served.

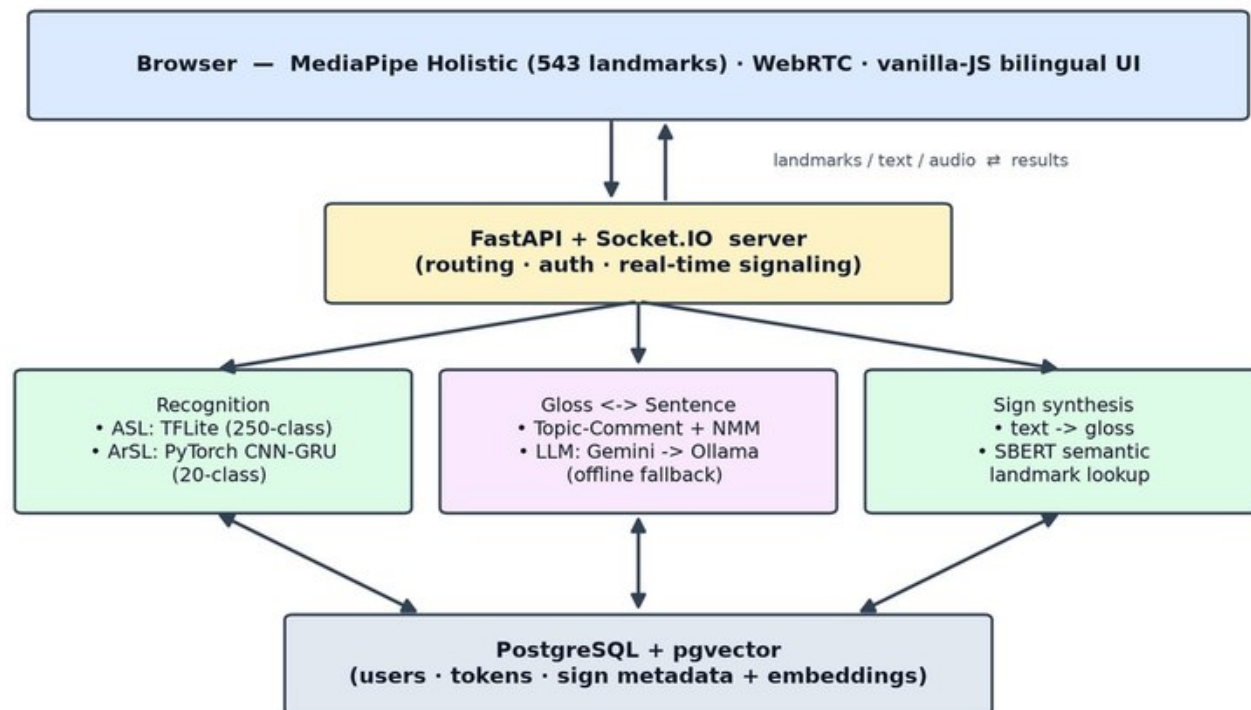
3 Special hardware

Gloves, depth cameras and multi-camera
rigs keep these tools in the lab.

Together: real-time, two-way, bilingual, in the browser

- **Zero install.** A webcam and a browser — nothing else.
- **Four directions.** Sign → text, sign → speech, text → sign, speech → sign.
- **Two languages.** ASL (250 signs) and Egyptian ArSL (20), each with its own model.
- **Live meetings.** Signer and speaker converse over WebRTC.

Figure 1.1 — High-level architecture of the Together system



Six objectives, all delivered

O1 Accessible front-end

Landmark capture in the browser — no special cameras or hardware.

O2 Robust recognition

Train and test ASL + ArSL models that generalize to unseen signers.

O3 Bridge the grammar gap

LLM turns recognized gloss into fluent sentences, with offline fallback.

O4 Reverse translation

Text and speech back into sign via semantic retrieval and an avatar.

O5 Unified experience

One bilingual web app, including a live two-person meeting mode.

O6 Measure the impact

Quantify recognition accuracy and the LLM's translation gain.

01

The Research Landscape

Datasets, recognition families, and five commercial systems — and the gap none of them fill.



The data landscape: rich for ASL, scarce for ArSL

Dataset	Language	Level	Size	Signers
WLASL	ASL	Isolated word	21k videos · 2,000 glosses	100+
MS-ASL	ASL	Isolated word	25k videos · 1,000 signs	222
AUTSL	Turkish SL	Isolated word	38k samples · 226 signs	43
How2Sign	ASL	Continuous	80+ hours	11
PHOENIX14T	German SL	Continuous	8,257 sentences	9
KArSL	ArSL	Isolated word	502 signs · 75k samples	3
ArSL2018	ArSL	Static alphabet	54k images · 32 classes	40

Recognition: from pixels to poses

Appearance-based

2D/3D CNNs (I3D) on raw video — strong but compute-hungry; $\approx 32\%$ top-1 on 2,000-sign WLASL.

Pose-based

Models on extracted skeleton keypoints match appearance models at a fraction of the cost.

Signer-independent proof

Skeleton-aware GCNs won the AUTSL challenge at $\approx 98\%$ — keypoints generalize across people.

Our choice

MediaPipe landmarks: compact, private, background-invariant — and they run in a browser.

Translation: gloss as the bridge

Neural SLT

Camgöz et al. formalized sign → sentence translation, with gloss as the intermediate representation.

Sign Language Transformers

Joint recognition + translation roughly doubled quality on PHOENIX14T.

The catch

End-to-end models need large aligned corpora of continuous signing — which do not exist for ArSL.

Our route

Isolated signs → gloss → a general-purpose LLM supplies the grammar. No parallel corpus required.

Five commercial systems — all one-directional

System	Direction	Languages	Hardware	Key limitation
SignAll	Sign → spoken	ASL	Multi-camera + gloves	Fixed rig; not portable
Hand Talk	Spoken → sign	ASL, Libras	None (mobile)	Cannot read signing
SLAIT	Sign → spoken	ASL	Webcam	Small vocabulary; pivoted to lessons
KinTrans	Sign → spoken	ASL, ArSL	3-D depth camera	Fixed installation
Signapse	Spoken → sign	BSL, ASL	None (pre-rendered)	Domain-limited output

The gap Together fills

Bidirectional

Both directions in one platform — plus a live meeting mode running them concurrently.

Bilingual

ASL and Egyptian ArSL as first-class languages, each with its own trained model.

Hardware-free

Webcam + browser only, built on public datasets — fully reproducible.

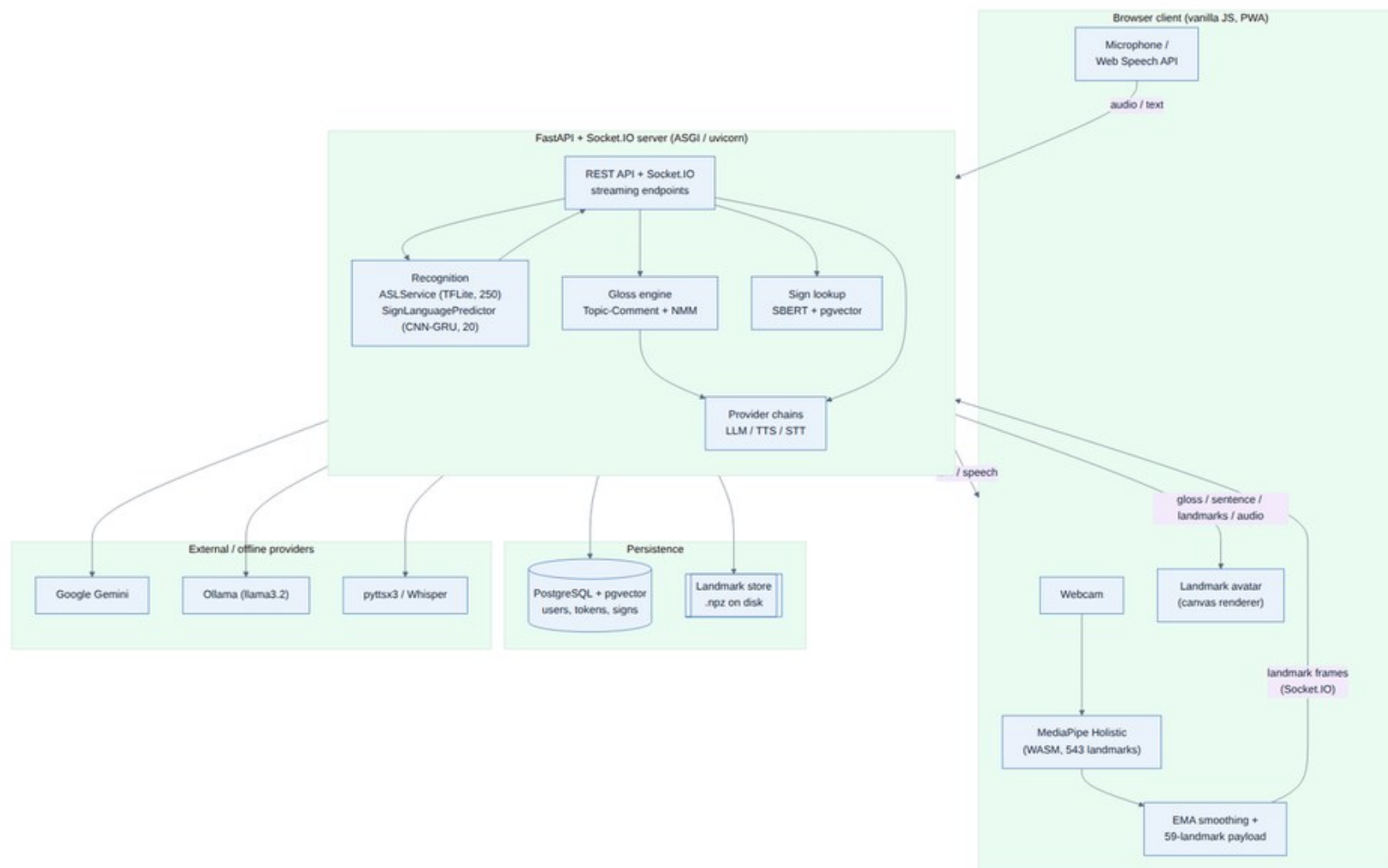
02

System Architecture

A browser client, a FastAPI + Socket.IO server, two recognition models, and a provider chain that degrades gracefully.



One architecture, four translation paths



The two pipelines, end to end

Figure 1.2 — The bidirectional translation pipelines

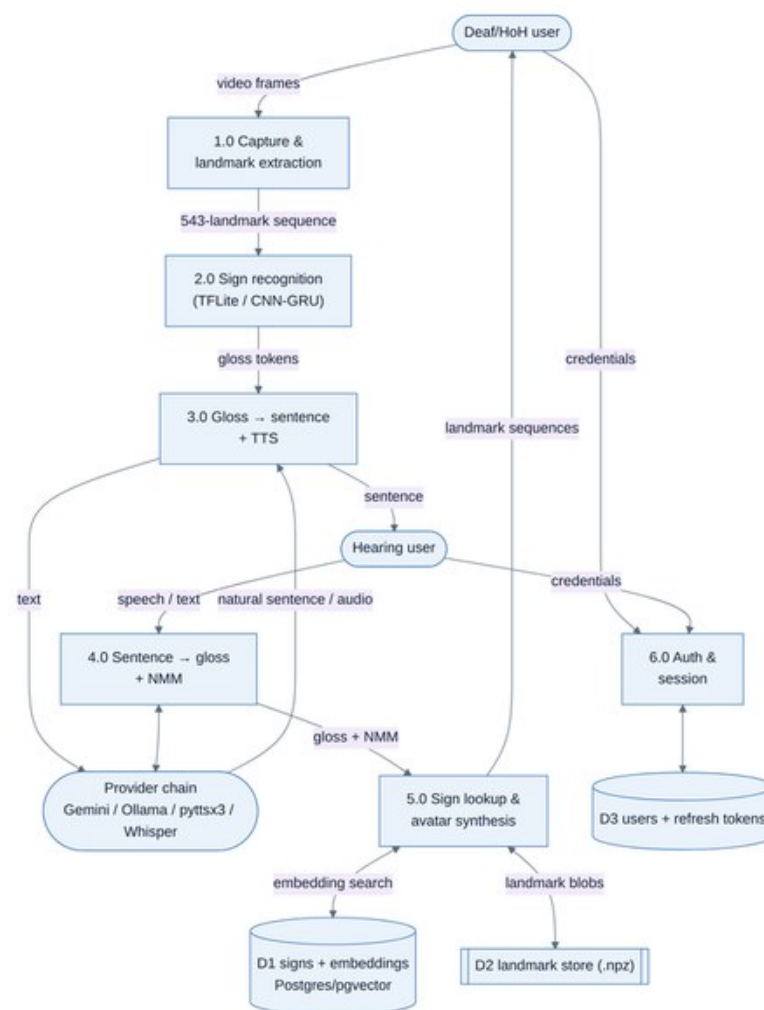
Sign -> Text / Speech



Text / Speech -> Sign

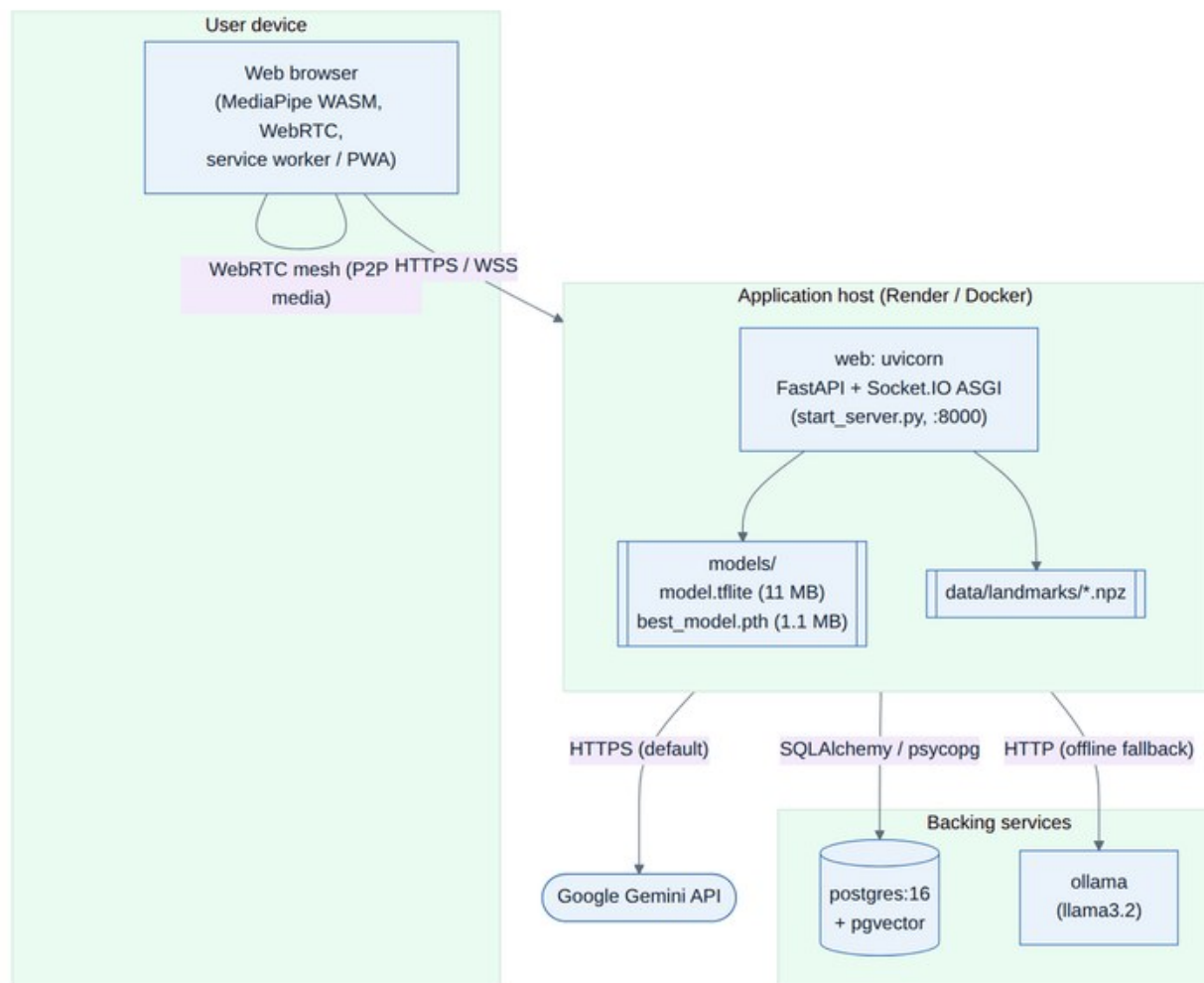


Data flow through six processes



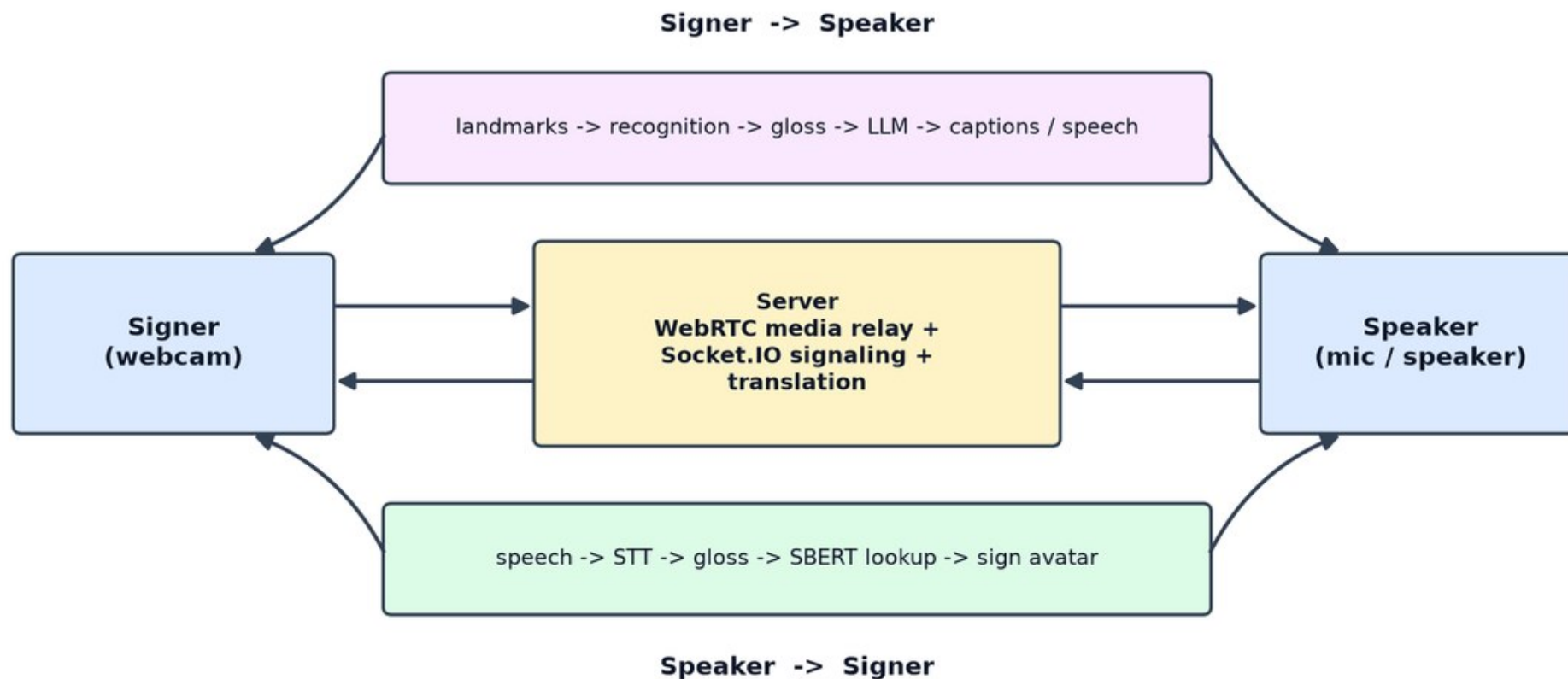
Deployment: CPU-only, container-first

- **Client.** Browser + WASM MediaPipe; installable as a PWA.
- **Server.** FastAPI + Socket.IO on uvicorn, Dockerized.
- **Data.** PostgreSQL 16 + pgvector; landmark store on disk.
- **AI providers.** Gemini primary; Ollama, pyttsx3, Whisper offline.
- **~No GPU anywhere.** Both models are quantized/optimized for CPU.



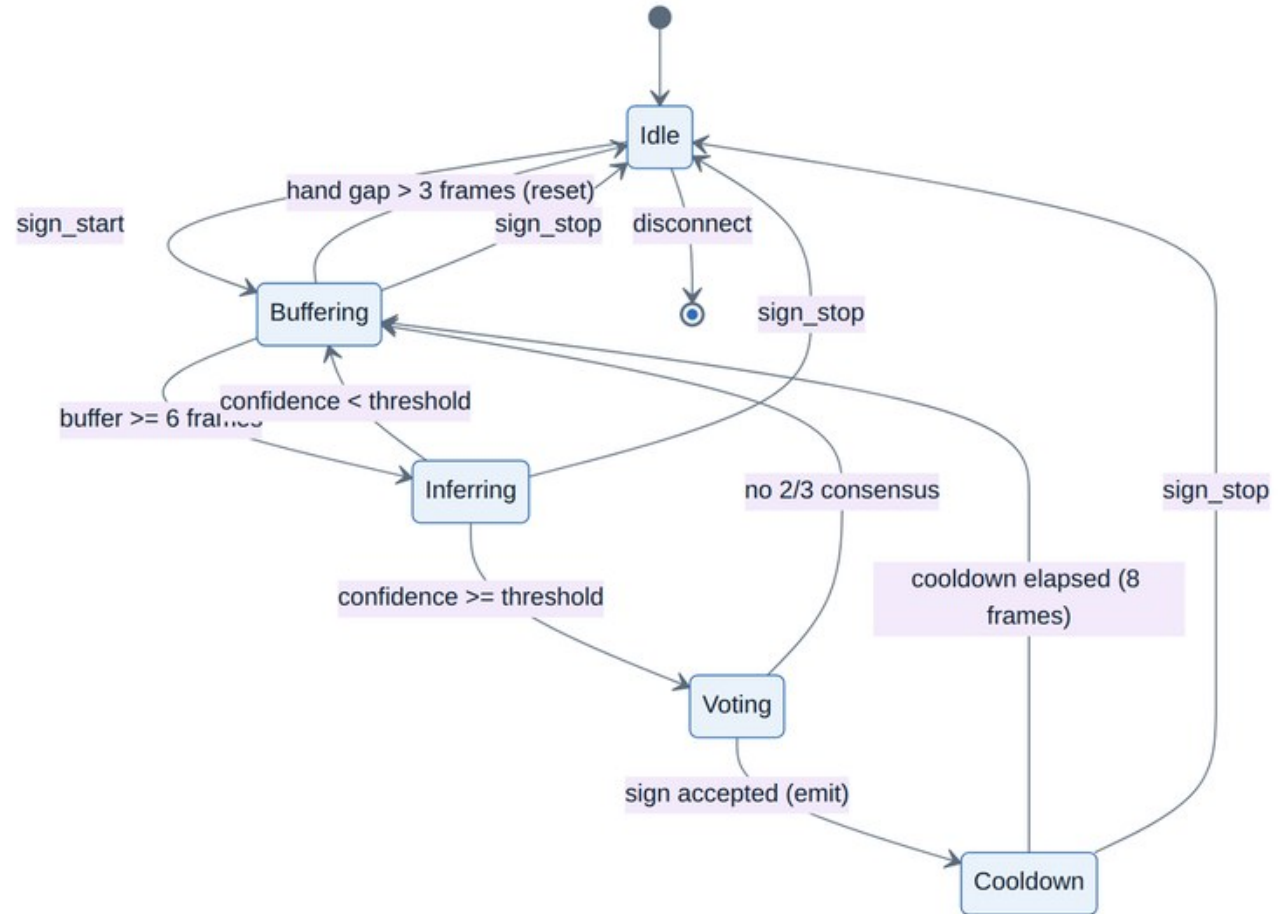
Live meetings: translation in both directions at once

Figure 1.3 — The live two-person meeting pipeline



Streaming recognition as a state machine

- **Rolling buffer.** Up to 60 frames of landmarks, streamed one frame per tick.
- **Vote + cooldown.** Majority vote over recent predictions; cooldown prevents repeats.
- **Hand-gap reset.** A pause flushes the buffer — the natural sign boundary.
- **~Result.** Noisy per-frame outputs become stable sign tokens.



03

The ASL Recognizer

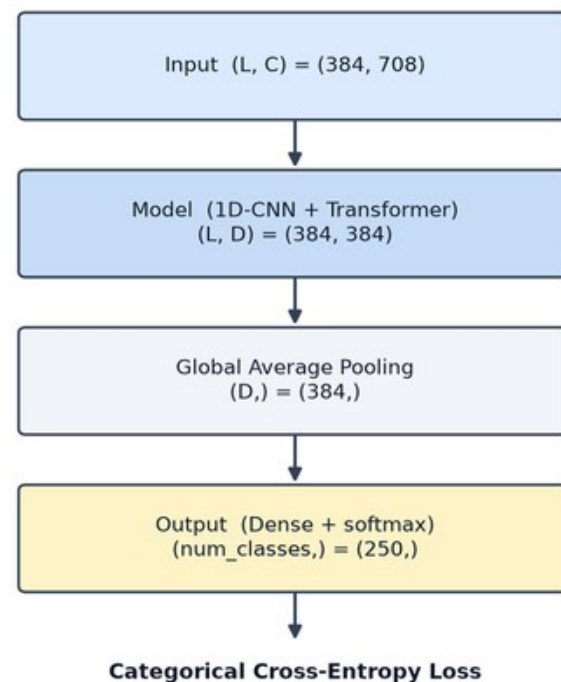
250 signs · Squeezeformer-style Conv1D + Transformer · trained on Google ISLR · exported to TFLite.



Google ISLR: the training corpus

- **Scale.** $\approx 100,000$ landmark sequences $\cdot$ 250 signs $\cdot$ 21 Deaf signers.
- **Our subset.** 4,078 sequences, split 70 / 15 / 15 for train, validation, test.
- **Landmark-native.** The dataset ships MediaPipe landmarks — no video processing.
- **~Deployed shape.** TFLite accepts raw (60, 543, 3) and preprocesses inside the graph.

Figure 4.1 — ASL model: input $\rightarrow$ output



training feature shapes; deployed TFLite
takes raw (60, 543, 3) and preprocesses in-graph

Preprocessing: 543 points → 708 features

- **Select & center.** 118 informative landmarks (lips, hands, upper body); drop z; center on the nose.
- **Normalize & differentiate.** Scale by per-clip σ ; add velocity Δ^1 and acceleration $\Delta^2 \rightarrow 118 \times 6 = 708$ features.

Figure 4.2 — ASL preprocessing (embedded in the TFLite graph)



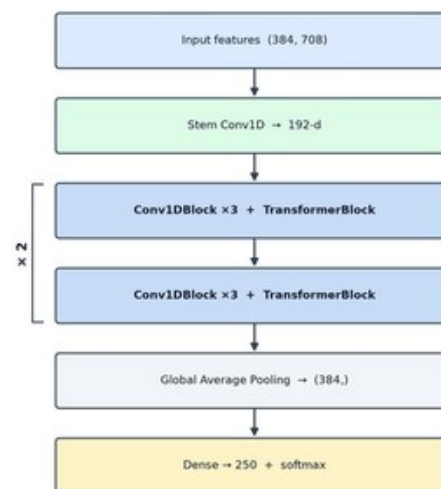
$$\tilde{p}_{t,i} = p_{t,i} - p_{t,\text{nose}}, \quad \hat{p}_{t,i} = \tilde{p}_{t,i} / \sigma, \quad \sigma = \sqrt{\frac{1}{|V|} \sum_{(t,i) \in V} \|\tilde{p}_{t,i}\|^2}$$

$$\Delta_t^{(1)} = \hat{p}_t - \hat{p}_{t-1}, \quad \Delta_t^{(2)} = \hat{p}_{t+1} - 2\hat{p}_t + \hat{p}_{t-1}, \quad f_t = [\hat{p}_t \parallel \Delta_t^{(1)} \parallel \Delta_t^{(2)}] \in \mathbb{R}^{708}$$

Macro architecture: Conv1D $\times$ 3 + Transformer, twice

- **Stem.** Linear projection of 708 features to a 192-d representation.
- **Body.** Two repetitions of three Conv1D blocks followed by a Transformer block.
- **Head.** Global average pooling $\rightarrow$ 250-way softmax.
- **~Lineage.** Adapts the Squeezeformer design of the top Google-ISLR solution.

Figure 4.3 — ASL macro architecture (Squeezeformer)



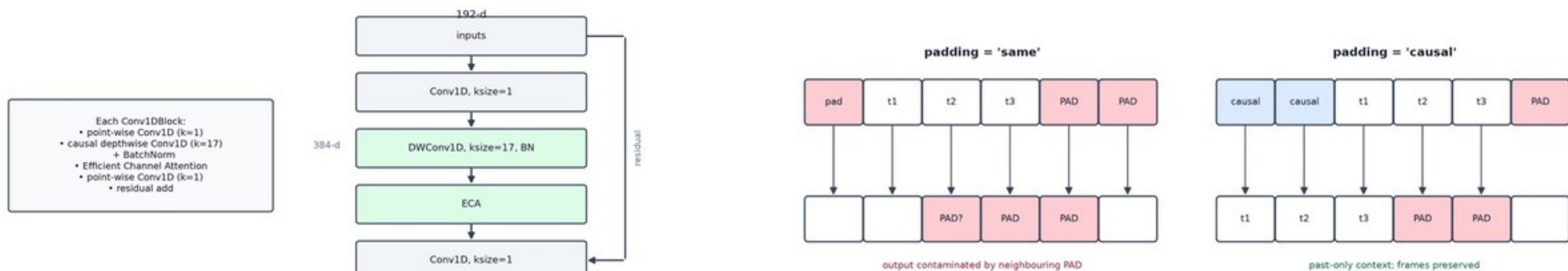
$$h_t^{(0)} = f_t W_{\text{stem}} + b_{\text{stem}}, \quad W_{\text{stem}} \in \mathbb{R}^{708 \times 192}$$

$$(60, 543, 3) \rightarrow (384, 708) \rightarrow (384, 192) \rightarrow \text{GAP} \rightarrow \mathbb{R}^{250}$$

Inside a Conv1D block — causal by construction

- **Structure.** Point-wise expansion $\rightarrow$ causal depthwise conv ($k=17$) $\rightarrow$ BatchNorm $\rightarrow$ channel attention $\rightarrow$ projection, wrapped in a residual.
- **~Causal padding.** No future frames leak into the past — the same network is safe for live streaming.

Figure 4.4 — Conv1DBlock internals



$$x' = x + \text{PW}_2(\text{ECA}(\text{BN}(\text{DW}_{k=17}(\text{swish}(\text{PW}_1(x)))))), \quad \text{swish}(x) = x \sigma(x)$$

Efficient Channel Attention: focus for almost nothing

- **Squeeze.** Global-average-pool each of the 384 channels to a single value.
- **Excite.** A 1-D convolution across channels yields sigmoid weights that rescale the features.

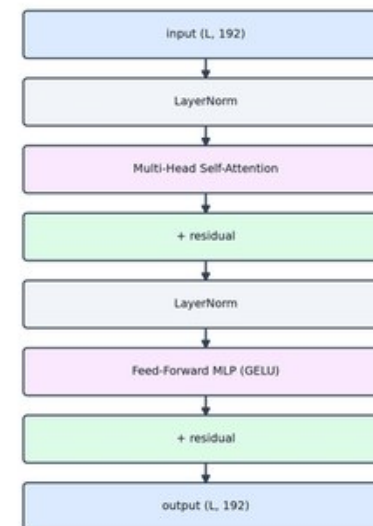


$$z_c = \frac{1}{T} \sum_{t=1}^T x_{c,t}, \quad a = \sigma(\text{Conv1D}_{k=5}(z)), \quad \tilde{x}_{c,t} = a_c x_{c,t}$$

Transformer block + a three-tower ensemble

- **Attention.** Multi-head self-attention: $H = 4$ heads, $d_k = 48$, pre-norm residuals.
- **FFN.** Two-layer MLP with swish activation.
- **~Ensemble.** Three independently-trained towers averaged at the logit level in the deployed TFLite.

Figure 4.6 — TransformerBlock



$$\text{Attention}(Q, K, V) = \text{softmax}\left(QK^\top / \sqrt{d_k}\right)V, \quad d_k = 48, \quad H = 4$$

$$\bar{l} = \frac{1}{3} \sum_{m=1}^3 l^{(m)}, \quad p = \text{softmax}(\bar{l}) \in \mathbb{R}^{250}$$

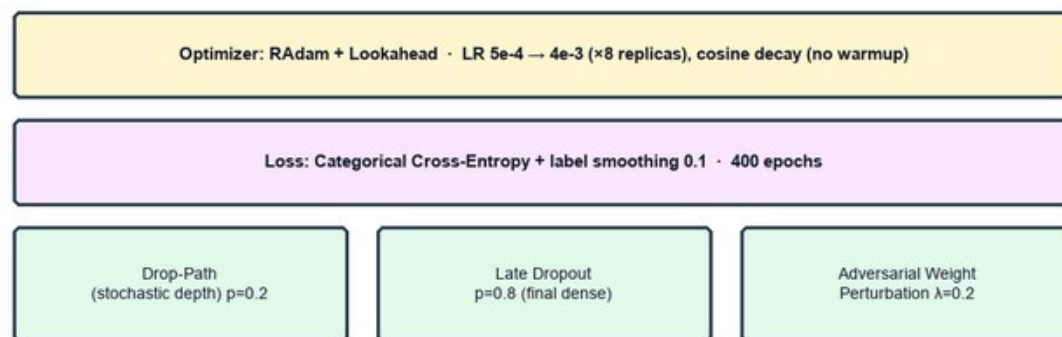
Augmentation: train for messy reality

- **Temporal.** Resample 0.5–1.5× for signing speed; mask 20–40% of frames for dropouts.
- **Spatial.** Horizontal flip for left-handed signers; affine transforms to $\pm 30^\circ$; random cutout.



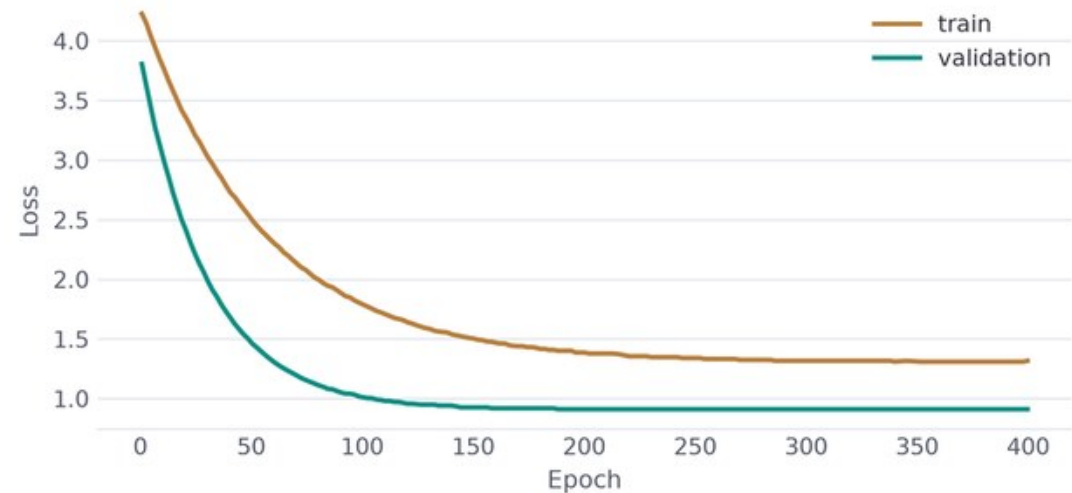
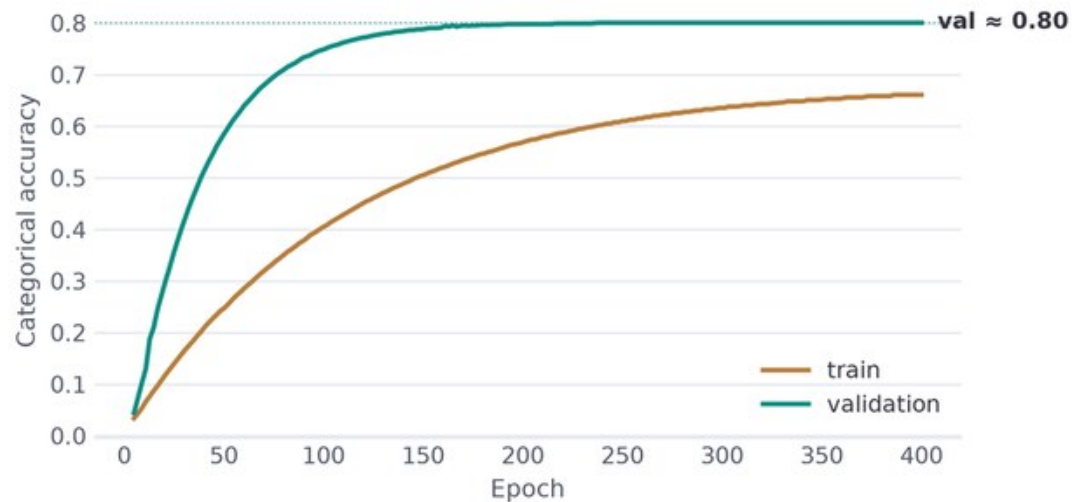
Training: heavy regularization for 250 classes

- **Optimizer.** RAdam + Lookahead, cosine decay from $4e-3$, 400 epochs, label smoothing $\varepsilon = 0.1$.
- **Regularizers.** Drop-Path 0.2 · late dropout 0.8 · Adversarial Weight Perturbation $\lambda = 0.2$.



$$\tilde{y}_c = (1 - \varepsilon) y_c + \varepsilon/K, \quad \varepsilon = 0.1, \quad K = 250, \quad \mathcal{L}_{\text{CCE}} = - \sum_{c=1}^K \tilde{y}_c \log p_c$$

Learning curves: validation reaches 0.80



04

The ArSL Recognizer

20 Egyptian signs · compact CNN + bidirectional GRU · built for a low-resource language.



Balaha ArSL-20: small, but many signers

8,437 samples

Phone-camera clips of 20 Egyptian signs — realistic, noisy capture conditions.

72 signers

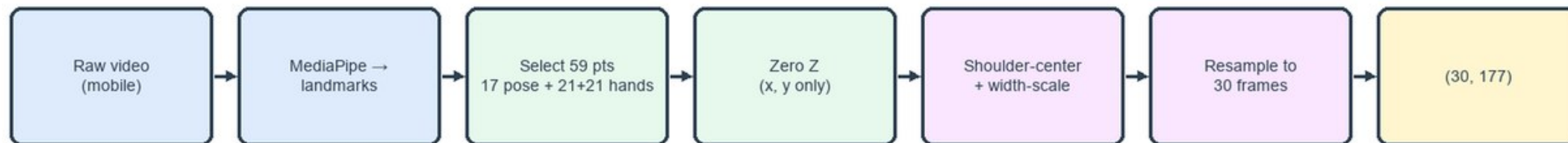
The real asset: enough signer diversity to hold entire people out of training.

70 / 15 / 15

Stratified split for training — plus a separate signer-independent protocol in the evaluation.

Preprocessing: invariance in four steps

- **59 landmarks.** 17 upper-body pose points + 21 per hand; z zeroed — phone depth is noise.
- **Normalize + resample.** Center on the shoulder midpoint, scale by shoulder width, resample to 30 frames $\rightarrow (30, 177)$.

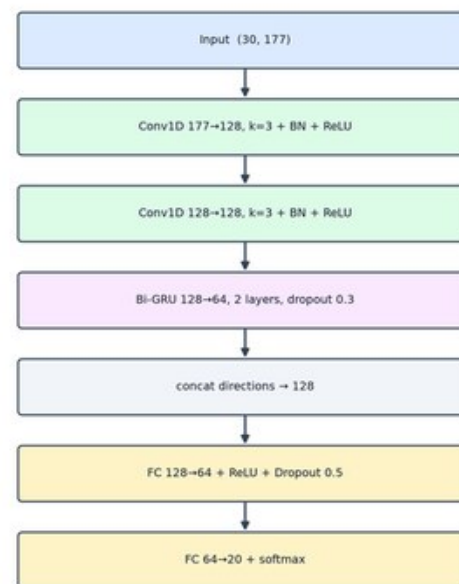


$$c = \frac{1}{2}(p_L + p_R), \quad s = \|p_L - p_R\|_2, \quad \hat{p}_{t,i} = \frac{p_{t,i} - c}{s}, \quad \tau_k = \left\lfloor \frac{k(T-1)}{29} \right\rfloor, \quad k = 0, \dots, 29$$

A compact CNN-GRU, sized to the data

- **Convolutions.** Two 1-D blocks
(177 → 128 → 128, k=3),
BatchNorm + ReLU.
- **Recurrence.** 2-layer BiGRU, 64
hidden units, dropout 0.3.
- **Head.** 128 → 64 → 20, dropout 0.5,
softmax.

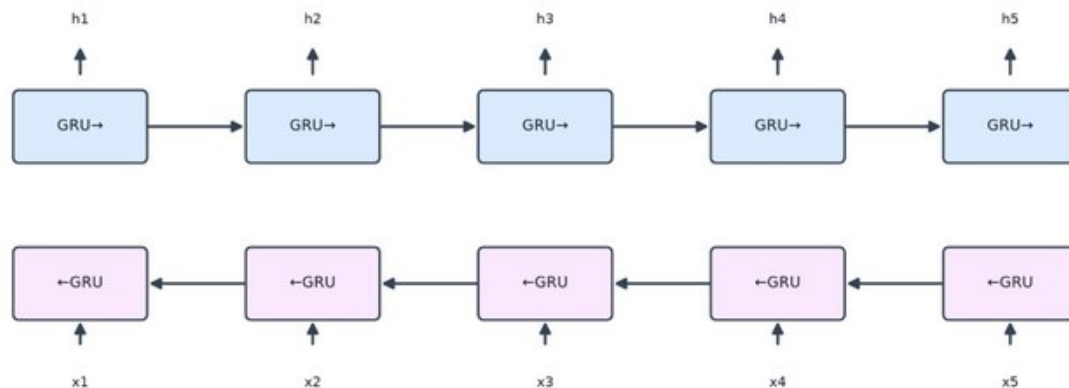
Figure 4.12 — ArSL CNN-GRU architecture



$$\bar{p} = \frac{1}{|\mathcal{W}|} \sum_{w \in \mathcal{W}} \text{softmax}(z^{(w)}), \quad \mathcal{W} = \{30, 45, 60\}, \quad \hat{y} = \arg \max_i \bar{p}_i, \quad \text{accept iff } \bar{p}_{\hat{y}} > \tau = 0.45$$

Why bidirectional recurrence

- **Full context.** Forward and backward passes are concatenated — every prediction sees the whole sign.



$$r_t = \sigma(W_{ir}x_t + W_{hr}h_{t-1} + b_r), \quad z_t = \sigma(W_{iz}x_t + W_{hz}h_{t-1} + b_z)$$

$$n_t = \tanh(W_{in}x_t + r_t \odot W_{hn}h_{t-1}), \quad h_t = (1 - z_t) \odot n_t + z_t \odot h_{t-1}, \quad h_t^{\text{bi}} = [h_t^{\rightarrow} \parallel h_t^{\leftarrow}]$$

Augmentation and training, tuned for small data

- **Online augmentation.** Mirroring 50% · inactive-hand masking 70% · affine 0.92–1.08× + Gaussian noise · temporal jitter.
- **Training.** Adam (wd 1e-4), lr 1e-3 with ReduceLROnPlateau, batch 64, ≤100 epochs, early stopping (patience 12).



Optimizer: Adam (weight decay 1e-4) · LR 1e-3, ReduceLROnPlateau (+2 after 3 stagnant epochs)

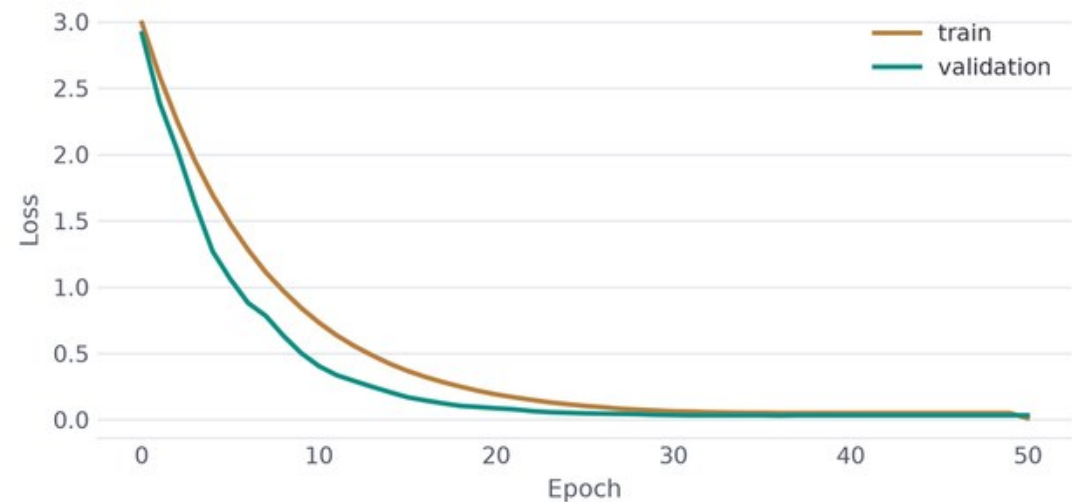
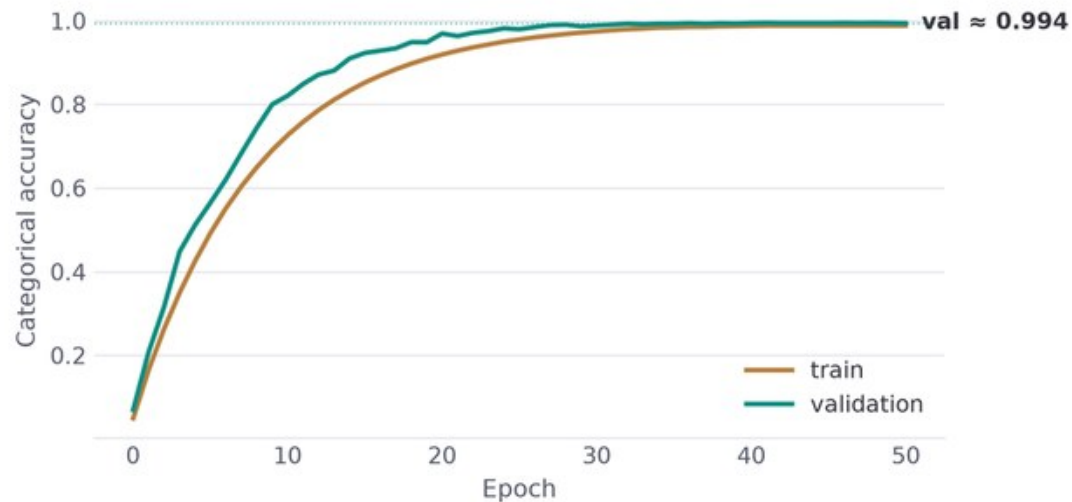
Loss: Cross-Entropy · 100 epochs, early stopping (patience 12) · batch size 64

GRU dropout
0.3

FC dropout
0.5

Mirror / mask /
affine / jitter

Learning curves: 99.41% on the test split



05

From Signs to Sentences

Gloss is not grammar. A language-model layer bridges Topic–Comment sign order and fluent sentences — both ways.



The grammar gap

Topic–Comment order

ASL fronts the topic: “STORE I GO” — not “I am going to the store.”

Dropped words

Articles and copulas simply do not exist in sign gloss.

Facial grammar

Questions and negation live on the face — non-manual markers.

An LLM supplies the grammar — with a safety net

1 Gemini 2.5 Flash

Few-shot prompts enforce
Topic–Comment ↔ SVO conversion in
both languages.

2 Bounded cache

256 recent gloss → sentence results —
repeated phrases cost ~0 ms.

3 Offline fallback

No cloud? Ollama serves a local LLaMA;
worst case, raw gloss is shown.

The reverse path — and facial grammar as data

Sentence → gloss

“What is your name?” becomes YOUR NAME WHAT — same few-shot layer, reversed.

NMM annotation

Sentence type is detected and mapped to markers: eyebrows furrowed, head forward.

Render-ready grammar

The avatar layer receives facial grammar it can act on — not just a word list.

Finding the right sign: semantic retrieval

Embed

Every stored sign carries a Sentence-BERT embedding (all-MiniLM-L6-v2, 384-d).

Search

pgvector HNSW cosine index inside PostgreSQL — a paraphrase still finds the sign.

Gate

Language-specific distance thresholds: 0.35 for English, 0.55 for Arabic.

Out of vocabulary? Spell it — smoothly

Fingerspell

Unknown words decompose into per-letter sign clips.

Bridge

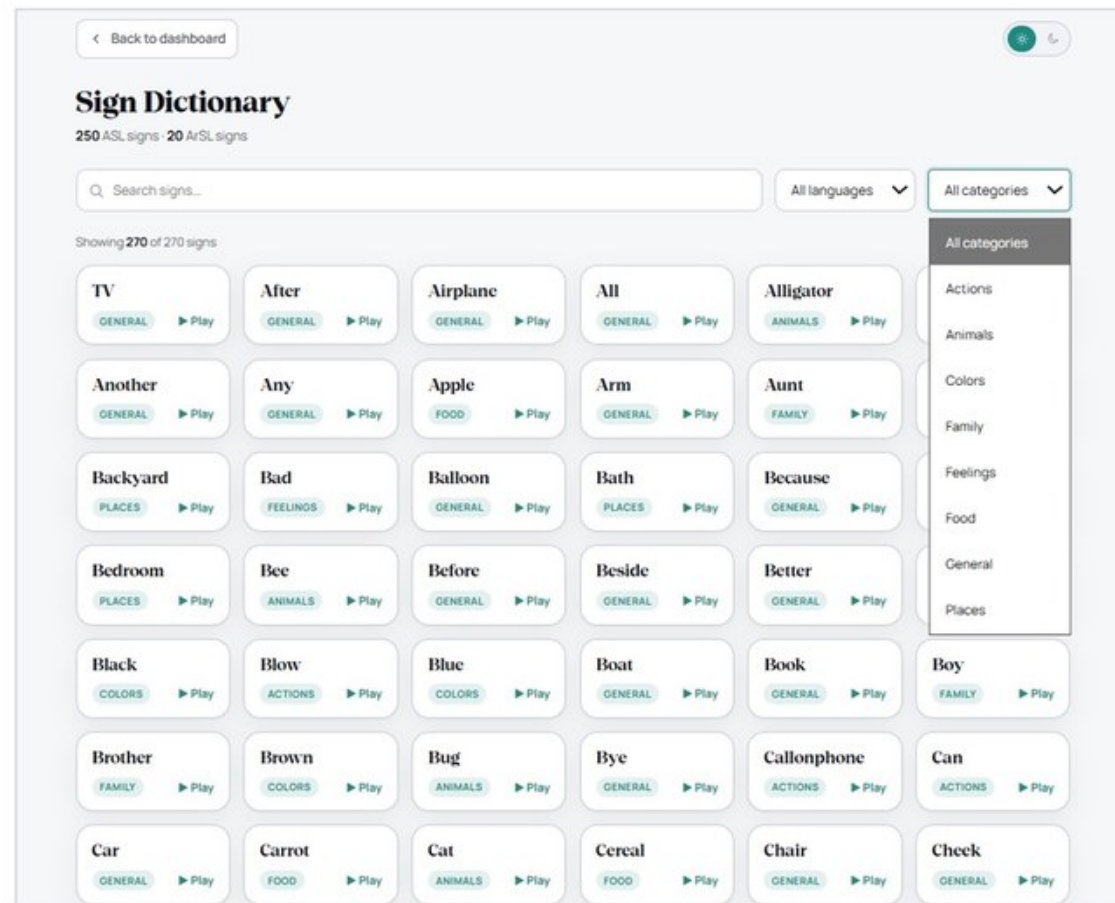
5 interpolated frames inserted at every clip boundary.

Smooth

A Savitzky–Golay filter (window 7) erases the stitch jerk, preserving each letter.

The avatar: landmarks back to motion

- **Canvas renderer.** Retrieved landmark sequences drive a skeleton avatar in the browser.
- **Reference videos.** Each dictionary entry pairs the avatar with the original human signer.
- **~Same store, both ways.** The landmarks we recognize from are the landmarks we animate with.



06

The Together Platform

Eight modules, two languages, one design system — installable as a PWA on desktop and mobile.



Eight modules

HandScript

Sign → text

VoiceBridge

Sign → speech

SignType

Text → sign avatar

TalkSide

Speech → sign avatar

SignLine

Live two-party meeting

Dictionary

Browse & search all signs

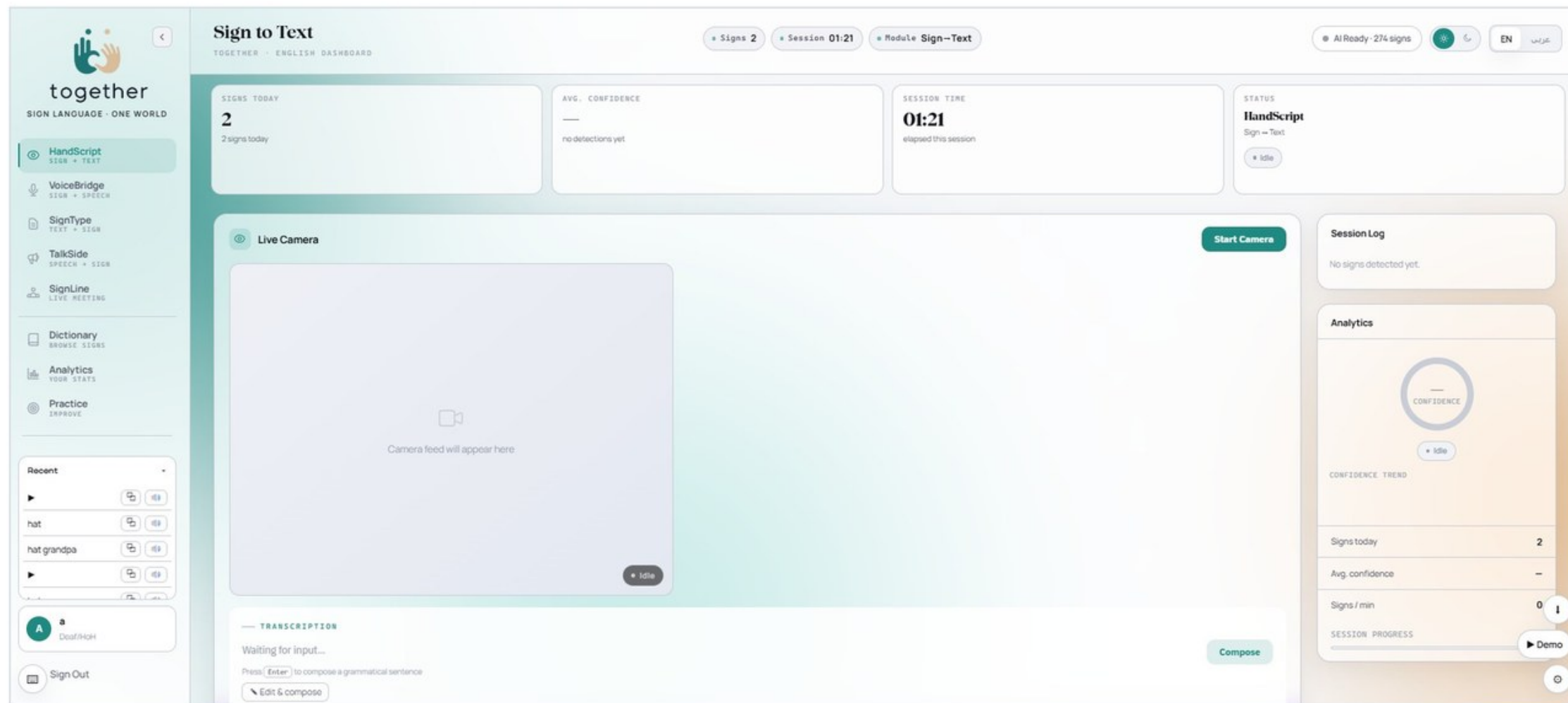
Practice

Avatar-prompted drills

Analytics

On-device session stats

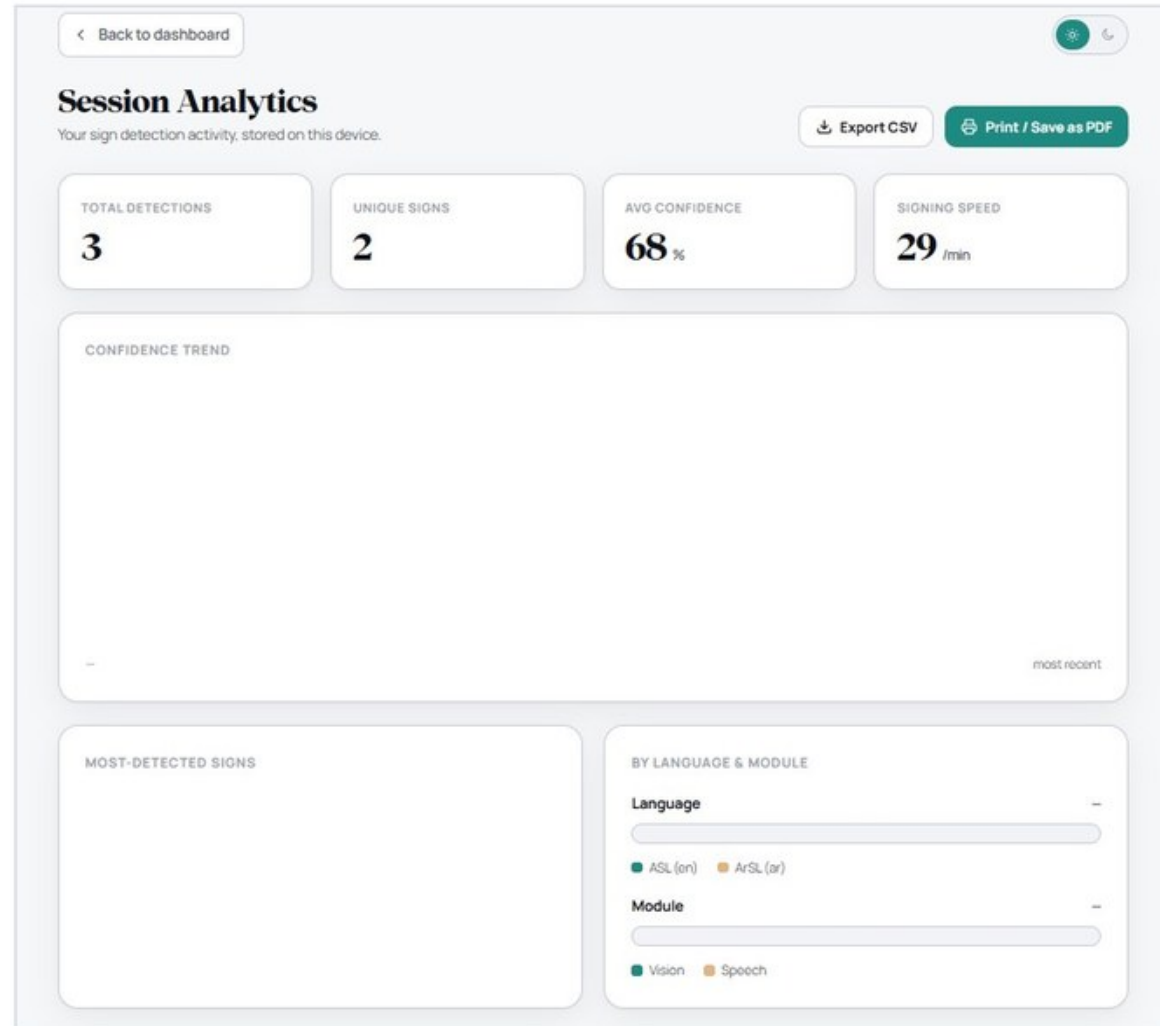
The shared real-time dashboard



Dictionary: avatar + human, side by side

The screenshot displays the 'Sign Dictionary' interface. At the top left is a 'Back to dashboard' link. The title 'Sign Dictionary' is followed by the count '250 ASL signs · 20 ArSL signs'. A search bar with the placeholder 'Search signs...' is present, along with dropdown menus for 'All languages' and 'All categories'. Below the search bar, it says 'Showing 270 of 270 signs'. The main area is a grid of sign cards. Each card shows a word (e.g., TV, After, Airplane, All, Alligator, Another, Any, Apple, Arm, Aunt, Backyard, Bad, Balloon, Bath, Because, Bedroom, Bee, Before, Beside, Better, Black, Blow, Blue, Boat, Book, Boy, Brother, Brown, Bug, Bye, Callonphone, Can, Car, Carrot, Cat, Cereal, Chair, Check) and a category tag (e.g., GENERAL, ANIMALS, FOOD, FAMILY, PLACES, FEELINGS, ACTIONS, COLORS). A 'Play' button is on each card. On the right, the 'All categories' dropdown is open, showing a list of categories: Actions, Animals, Colors, Family, Feelings, Food, General, and Places.

Analytics: insight without surveillance



Practice: drilled by the real recognizer

Practice Mode

We show a target sign — sign it back to the camera and we grade your attempt live.

ASL (English) ▼

New round

ATTEMPTS
0

CORRECT
0

ACCURACY
—

STREAK
0

TARGET SIGN

1 OF 10

Tiger

ANIMALS

TIGER



Show me

Skip —

YOUR ATTEMPT



Start the camera, then sign the target word.

Idle · heard "tree"

Start camera

Next —

THIS ROUND

0 / 10 COMPLETE

Tiger

Bird

Frog

Kiss

Wake

Cry

Pajamas

Clean

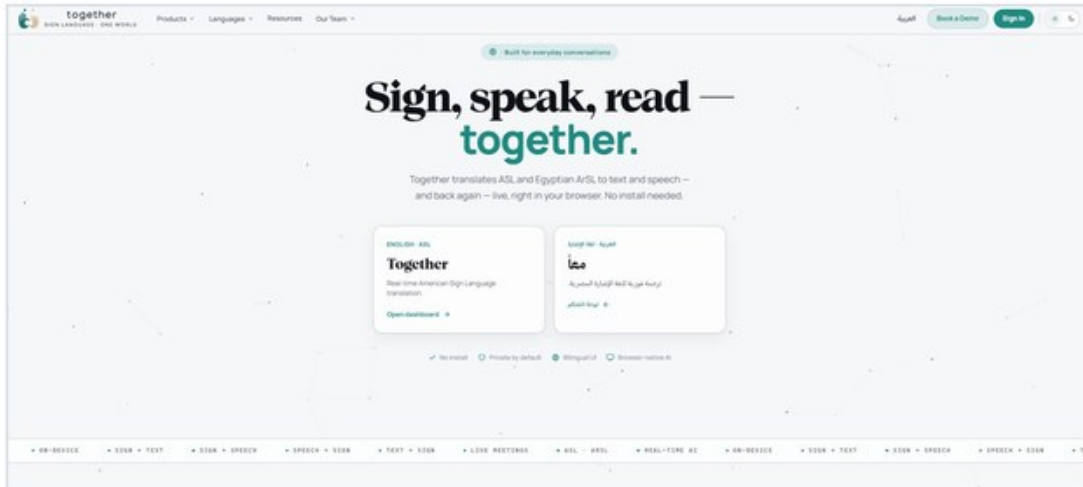
Cheek

Scissors

THE PLATFORM

Fully bilingual, fully responsive

- **True RTL.** The Arabic interface is a complete right-to-left mirror — every label, chart and control.



Security & privacy, end to end

Argon2id

Memory-hard password hashing; only hashes are stored.

Rotating JWT

Short-lived access tokens; refresh tokens hashed, expiring, rotated on use.

WhatsApp OTP

Registration verified by a one-time code — accounts tie to a real number.

Rate limiting

Sliding-window limits blunt credential stuffing.

CORS + headers

Restrictive origin policy and standard hardening headers.

Landmarks only

Raw video never leaves the browser — recognition sees coordinates.

07

Does It Work?

Recognition, generalization, translation quality against human references, and latency — with baselines.



Four families of evidence

Recognition

Accuracy, macro precision / recall / F1, confusion analysis — on held-out test sets.

Generalization

ASL: cross-dataset on SignASL. ArSL: held-out signers never seen in training.

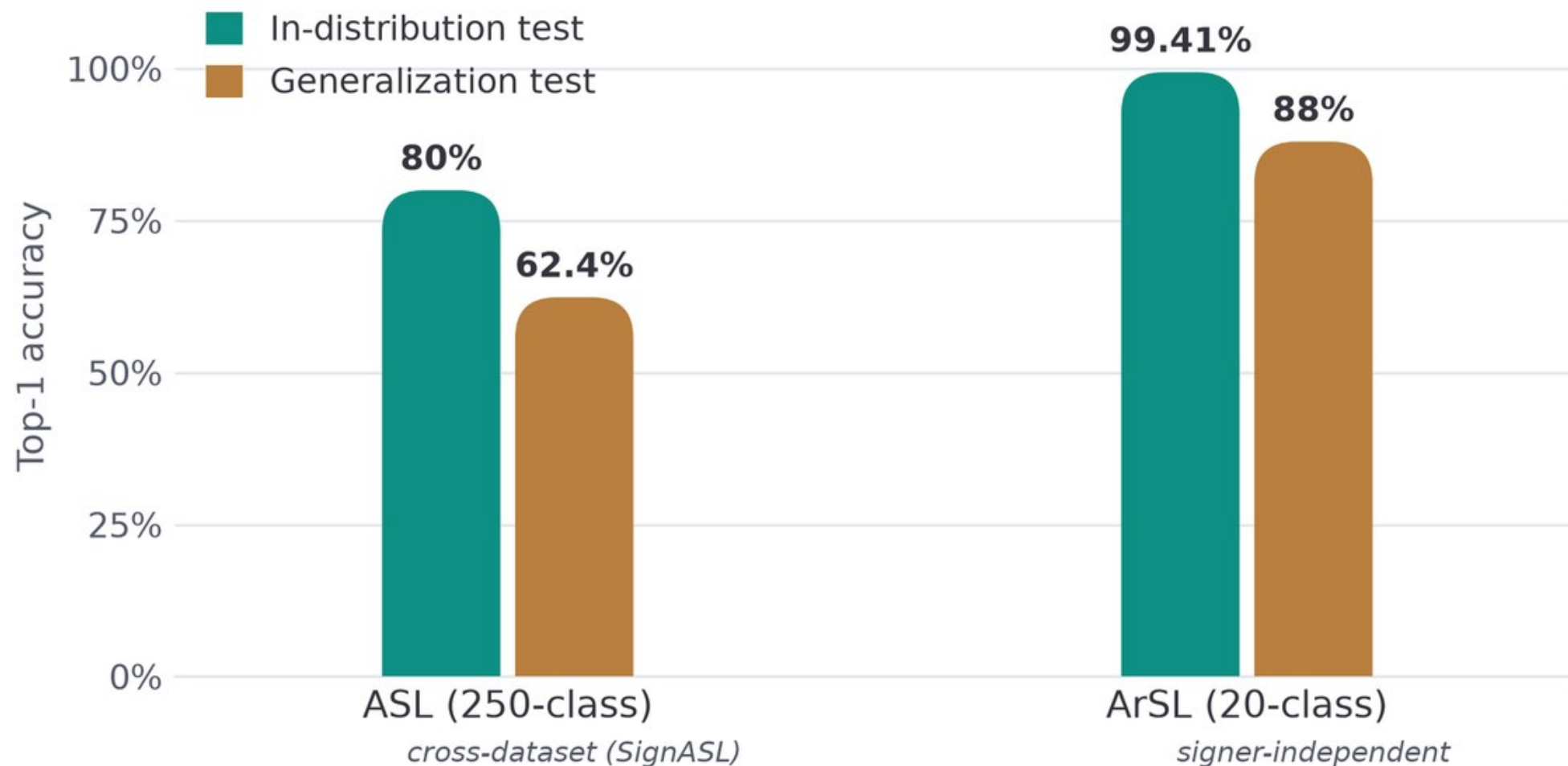
Translation quality

BLEU and chrF versus human references, always against a raw-gloss baseline.

System performance

Per-model inference latency and the effect of INT8 quantization on CPU.

Recognition: in-distribution vs. the honest number

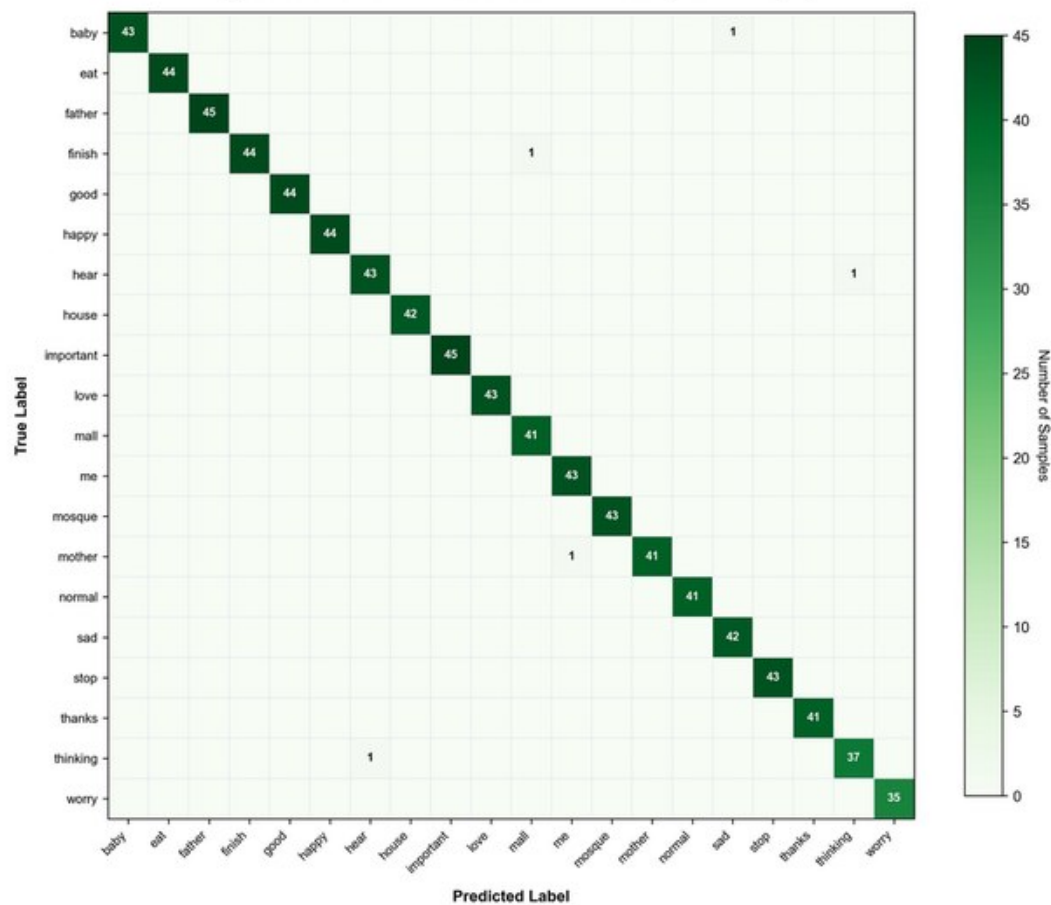


Per-model metrics on held-out test sets

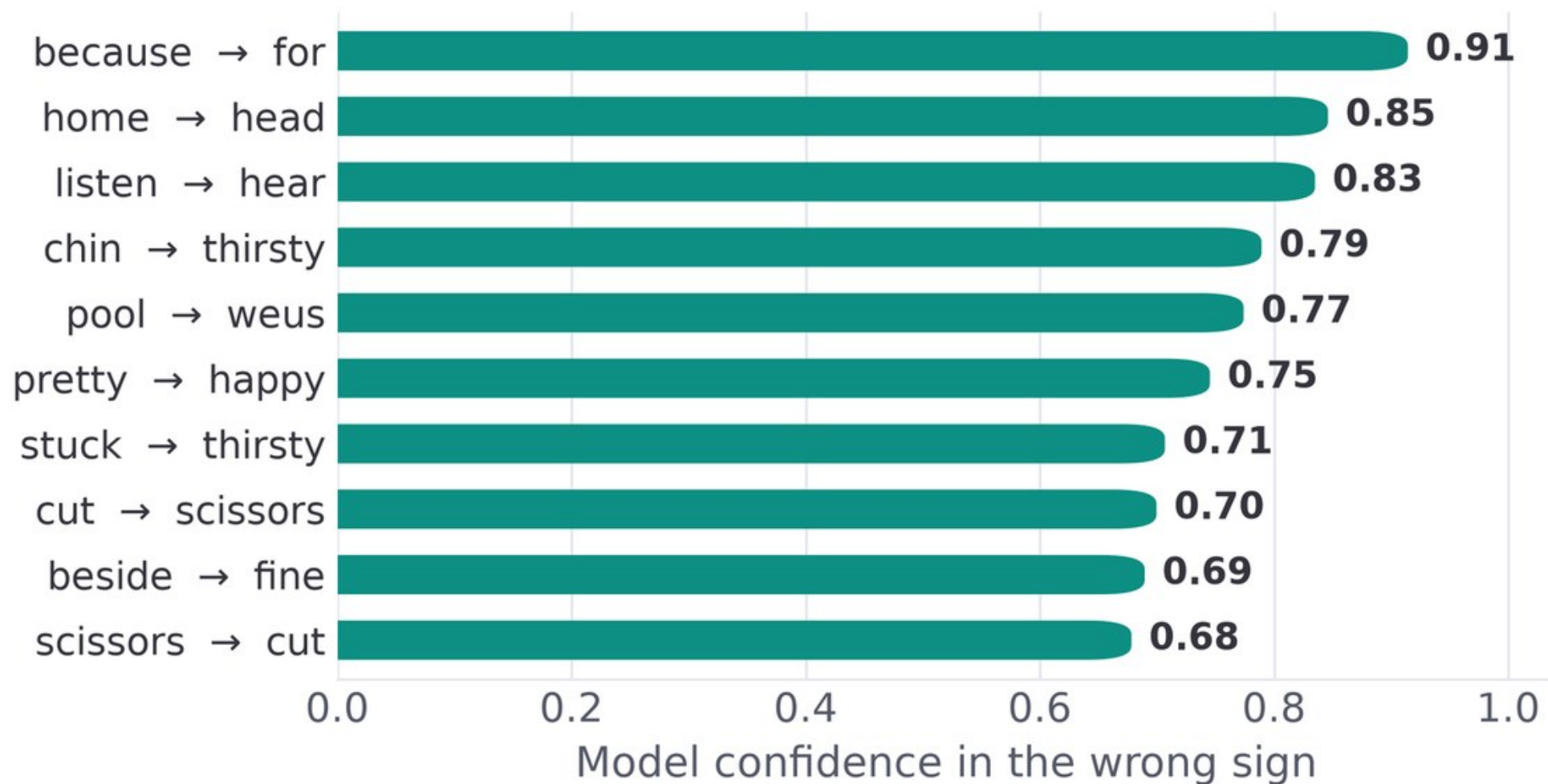
Metric	ASL (250-class)	ArSL (20-class)
Top-1 accuracy	80.00%	99.41%
Top-5 accuracy	95.00%	100.00%
Macro precision	0.81	0.99
Macro recall	0.80	0.99
Macro F1	0.80	0.99
Cross-entropy loss	0.90	0.03

ArSL: where the 0.6% lives

Figure 5.5 — ArSL Model 20×20 Confusion Matrix (Validation Split)



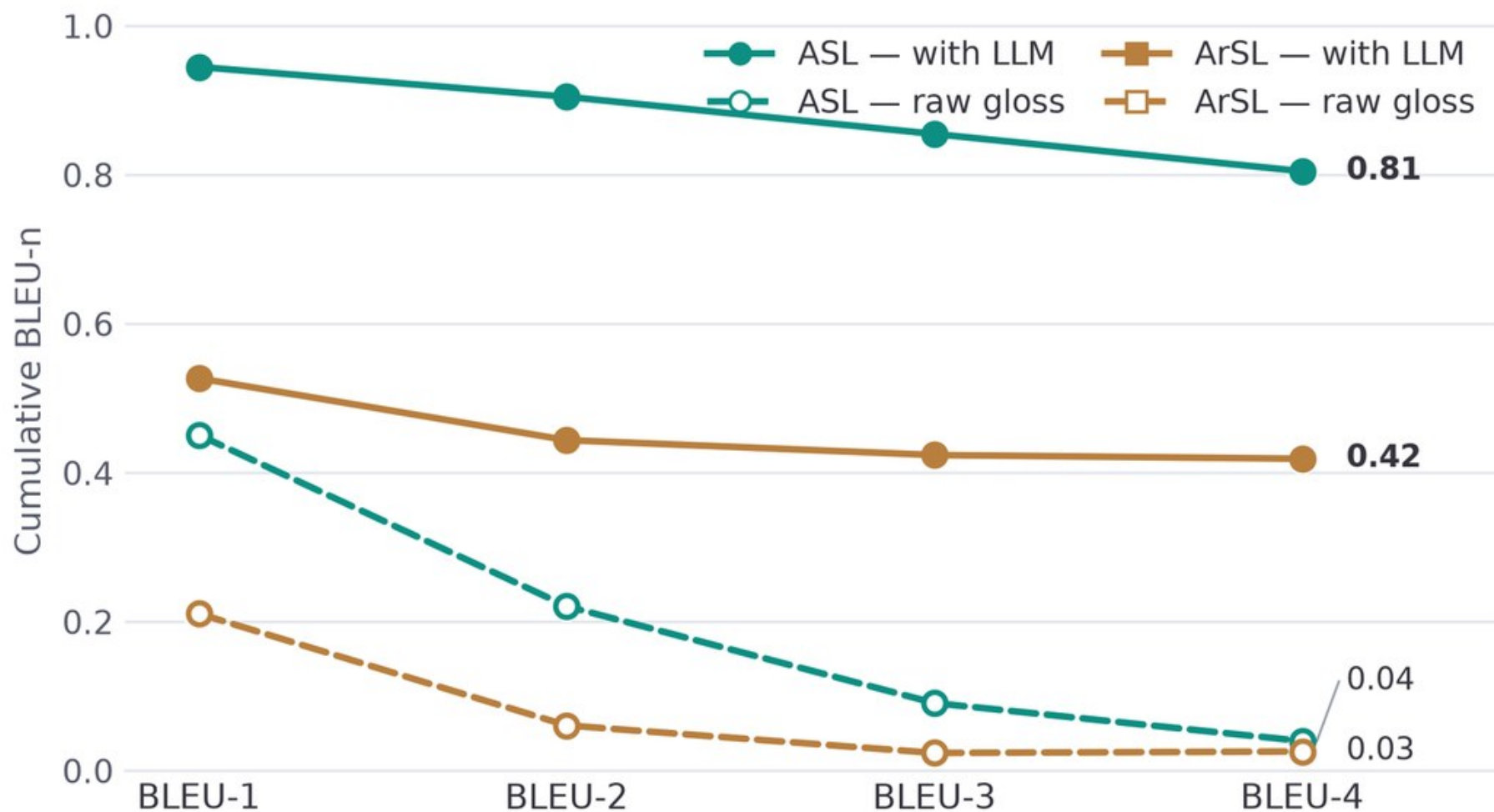
ASL: which signs fool the model — and why



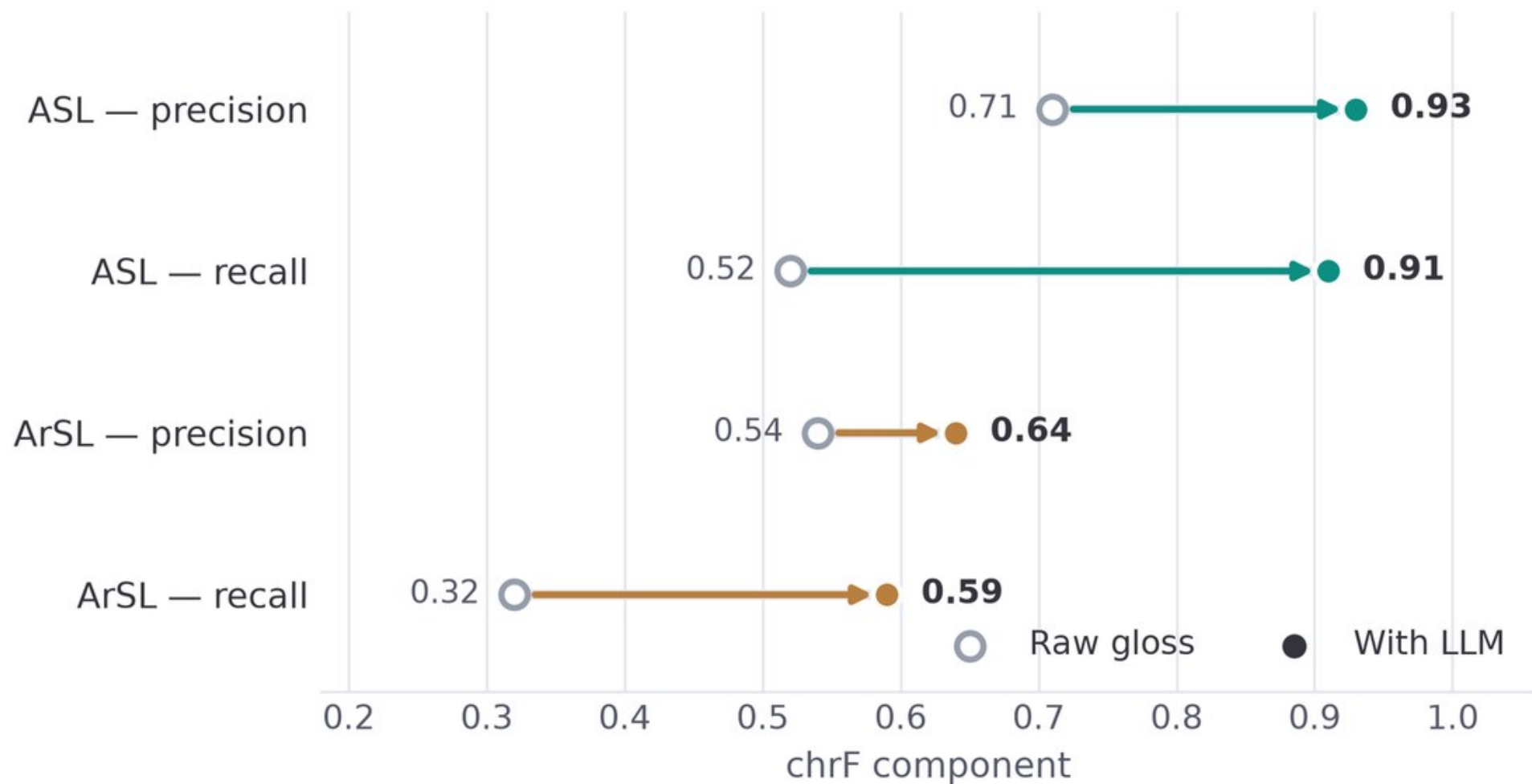
Translation quality: the LLM's contribution, isolated



BLEU-n profiles: fluency, not just words



Precision vs. recall: the LLM supplies grammar



Fast enough to feel live — on CPU

43.5 ms

ASL inference, average (58.7 ms max)

4.2 ms

ArSL inference after INT8 —
1.78× faster than fp32

~0 ms

Repeated gloss → sentence
via the 256-entry cache

0 GPUs

TFLite XNNPACK +
quantized PyTorch, 2–4 GB
RAM total

Limitations we want on the record

Isolated signs, not continuous

The LLM composes sentences from isolated signs; natural conversational signing is future work.

ASL split is random

The 80% is not signer-independent; the 62.4% cross-dataset result is the honest proxy.

20-word Arabic vocabulary

A real contribution to a low-resource language — but far from practical coverage.

20 reference sentences

BLEU on a small set is high-variance; chrF and human judgement carry more weight.

Each limitation maps to a next step

Isolated → **continuous**

Sequence-to-sequence recognition over the same landmark stream.

Random → **signer-independent**

Re-train and re-evaluate ASL under held-out signers, as done for ArSL.

20 → **hundreds of signs**

A documented, consented Egyptian ArSL collection effort.

Metrics → **people**

A user study with Deaf and hard-of-hearing participants on real tasks.

Four contributions

1 A bidirectional, bilingual, browser-based translator

Four directions plus live meetings — no gloves, no depth cameras, no installation.

2 A working Egyptian ArSL recognizer

88% signer-independent — a concrete contribution to an under-resourced language.

3 Gloss-mediated, LLM-assisted translation

Fluent output without the parallel corpora end-to-end models require — chrF 0.54 → 0.91.

4 A reproducible engineering blueprint

Public datasets, CPU-only inference, graceful offline degradation.

Where this goes next

Continuous signing

From isolated signs to sentence-level recognition.

Bigger ArSL

Expand the Egyptian vocabulary; document and consent the corpus.

Signer-independent ASL

The same honest protocol we applied to Arabic.

User study

Deaf participants, real tasks, comprehension and usability.

Edge deployment

Recognition fully in-browser — cut the server round-trip.

More languages

The architecture already spans two typologically different ones.



**“An accessible, bidirectional, bilingual
sign-language translator is achievable today.”**

Commodity hardware. Public datasets. A browser. Together is our proof — and our contribution to technology that serves under-represented Deaf communities.

Live Demo

1 · HandScript: live ASL sign → sentence → speech. 2 · SignType: type a sentence → the avatar signs it. 3 · Arabic dashboard (RTL) → ArSL recognition. 4 · SignLine: two-device live meeting.



Thank you.

Questions welcome — on the models, the language layer, the platform, or the evaluation.

Abdelfattah Moustafa · Michael Abdallah · Ahmed Mohamed Nagib · Ahmed Ashraf Shawareb — Supervisor: Prof. Medhat Awadallah

Full training configurations

Hyperparameter	ASL (Squeezeformer-style)	ArSL (CNN-BiGRU)
Dataset / classes	Google ISLR — 250 signs	Balaha ArSL-20 — 20 signs
Input tensor	(384, 708) train · raw (60, 543, 3) deployed	(30, 177)
Optimizer / LR	RAdam + Lookahead · $5e-4 \rightarrow 4e-3$ cosine	Adam (wd $1e-4$) · $1e-3$ ReduceLROnPlateau
Epochs / batch	400	≤ 100 , early stop (pat. 12) · batch 64
Loss	CCE + label smoothing $\varepsilon = 0.1$	Cross-entropy
Regularization	Drop-Path 0.2 · dropout 0.8 · AWP $\lambda = 0.2$	Dropout 0.3 (GRU) / 0.5 (dense)
Inference gate	$\tau = 0.80 + 15\text{-vote majority}$	$\tau = 0.45 + 30/45/60$ window ensemble
Export	TFLite (3-tower ensemble, XNNPACK)	PyTorch, INT8 dynamic quantization